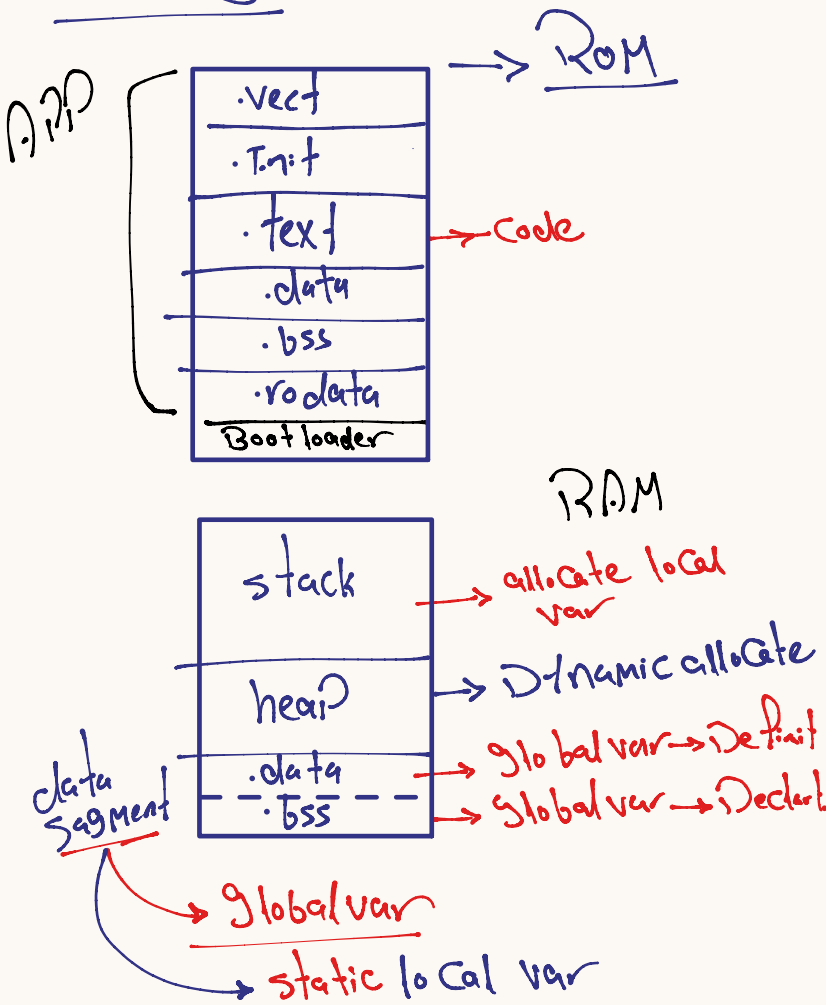


* Memory



* static allocate

→ allocate fixed size in

RAM

allocate during Runtime (init) | * Size known before Run time

* Dynamic allocate

* During Runtime → x Size

* use function to Request allocate

→ lib → #include <stdlib.h>

malloc calloc realloc free

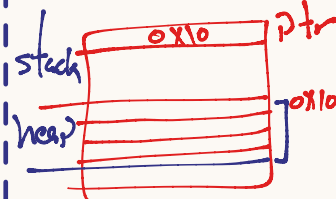
1) malloc :- Memory allocate

→ inputs → No of byte

→ Return → void pointer

Address first byte null

u8 * ptr = (u8 *) malloc (5);



for (i=0; i<5; i++)

{ scanf ("%d", ptr+i);

scanf ("%d", ptr[i]);

lab 1:

2) Calloc

→ inputs → ① Number of blocks

→ ② Size of each block

→ Return → void pointer

first Address null

u16 * ptr = (u16 *) calloc (4, 2)

u8 * ptr = (u8 *) malloc (8);

for (i=0; i<8; i++)

{ printf ("%d\n", ptr[i]);

* Calloc → clear bit zero for each byte



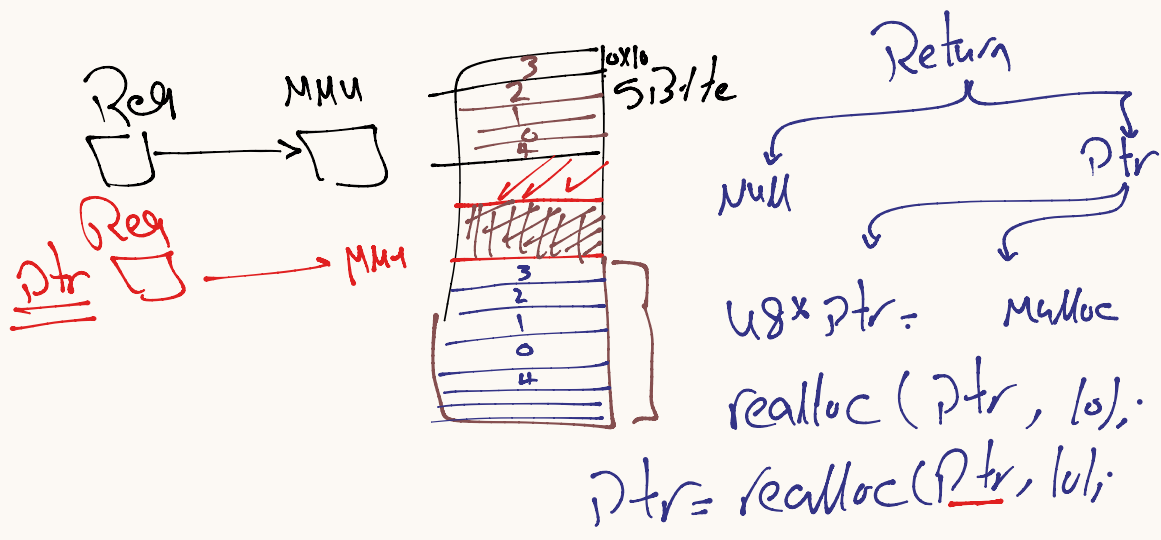
realloc →

3) realloc :-

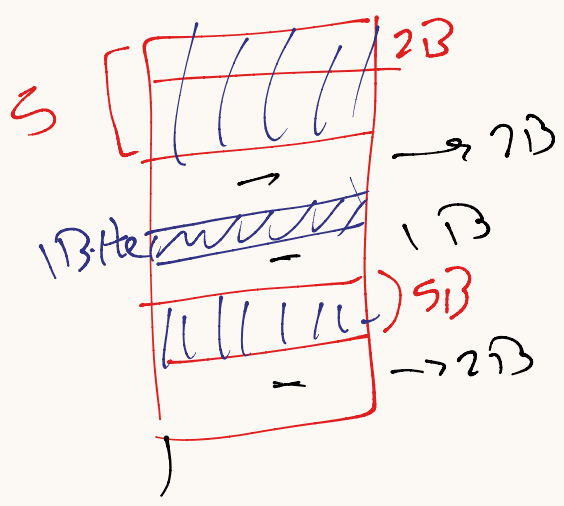
→ input → pointer

→ new total size

→ return: → pointer

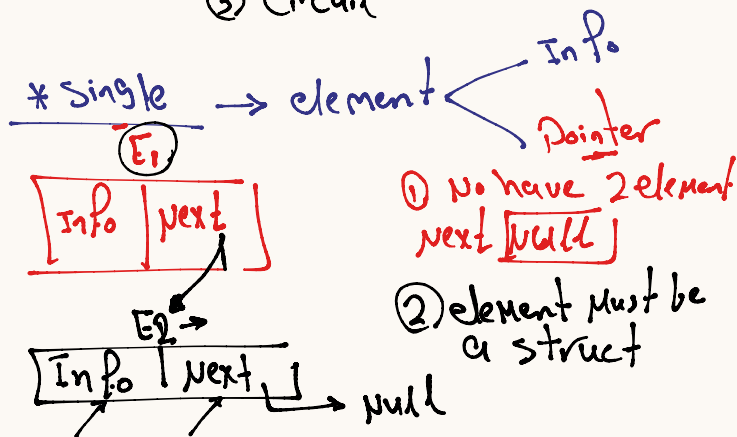


malloc(5)



* free (

* linkedlist ① single ② Double ③ Circular



state 1 → create struct

```

typedef struct single
{
    uint8_t Info;
    struct single* next;
} single_linked_t;

```



single_linked_t * start = NULL;

① Not have and list start
NULL → ~~Info~~ Next

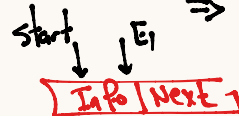
② create first element

single_linked_t * E1
= (single_linked_t*) malloc(sizeof)

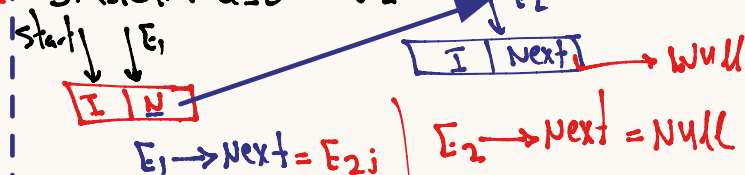
single_linked_t * E1 = (single_linked_t*) malloc(sizeof (single_linked_t))

start = E1;

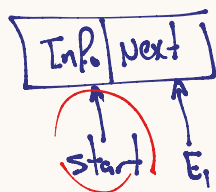
start → E1 ⇒ E1 → next = NULL;



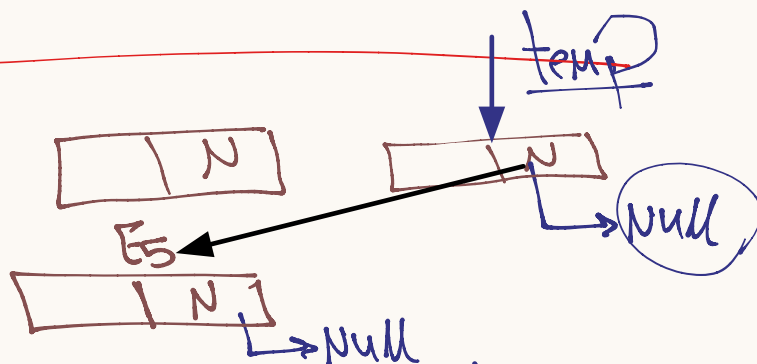
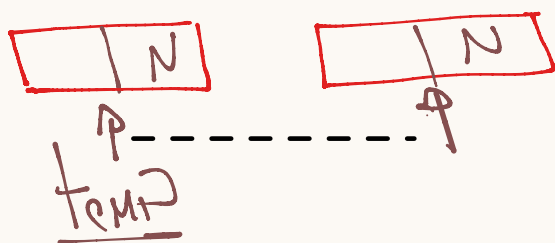
single_linked_t * E2



* Add Element from start

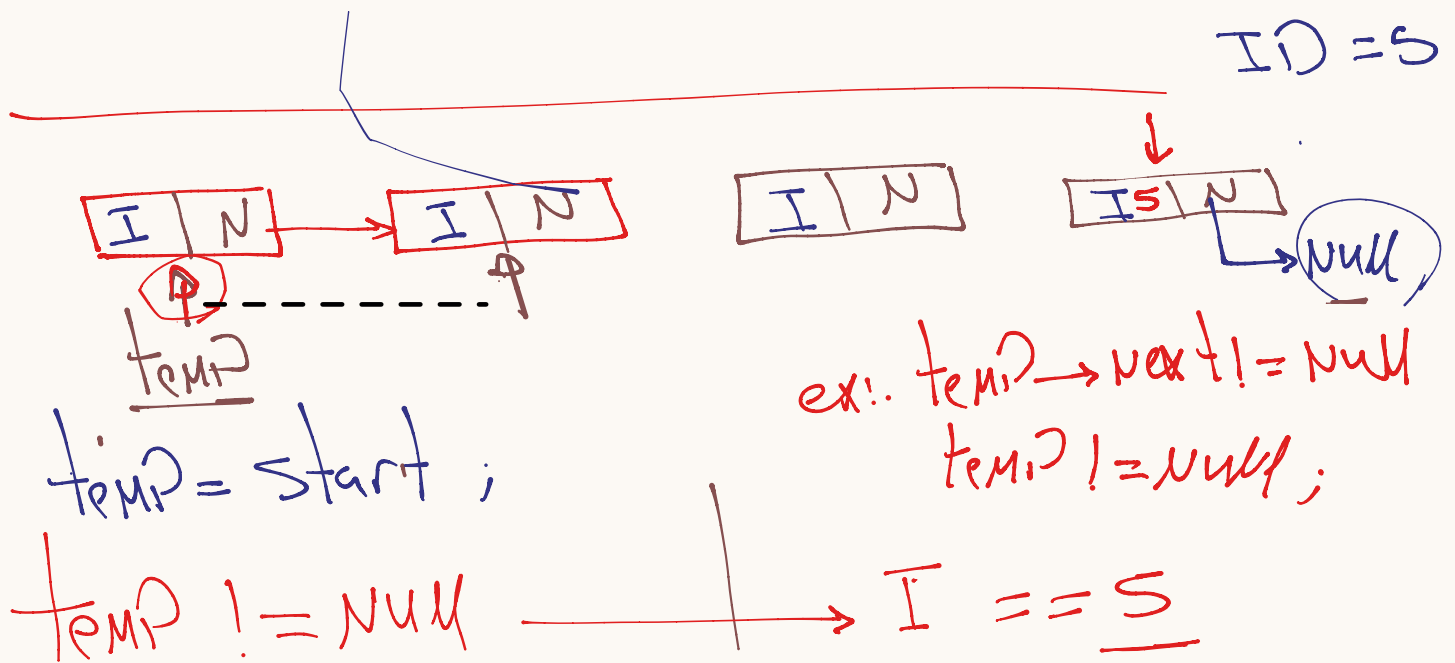


E2 → next = E1;
start = E2;



temp → next != NULL;
temp = temp → next;

* E5 → next = NULL;
* temp → next = E5;



```

while (temp != null)
{
    if (temp → I == 5)
    {
        Action
        temp = temp → next
    }
}
  
```